__/__/____

* System design

⊙ What is low-level Design (LLD)?

↳ Design the internal structure ("skeleton") of an application by identifying class/objects, their relationships, data flows, and how DSA solutions are plug into the structure

↳ DSA : Solves isolated problem (eg: finding shortest path in an array/graph) using algorithm like binary search, quicksort etc.

↳ LLD : Determines which objects exists in the system and how they interact, then applies DSA inside the structure

⊙ Idea through Story → App

- Two approaches to build "Quick Ride"
- Scenario : Build a ride-booking app ("QuickRide") like Uber/Ola

Student-A : Approach - DSA First

1. Problem decomposition
   - → Map city intersection to graph nodes, road to edges
   - → Use Dijkstra's algo to compute shortest route
   - → Use min-heap (priority queue) to match riderers to closest drivers.

2. Gap → No identification of classes/entities (Users, Rider location, Notification, Payment)
   - → Omits data security (masking phone no)
   - → Missing integration points (Notification, Payment gateway)
   - → No consideration for scaling to millions of users.

"If DSA is the brain, LLD is the skeleton of your application."

__/__/____

Student-B : LLD-first Approach

- Entity identification
  - ↳ Objects: Users, Rider, locations, NotificationServices paymentGateway, etc
- Define relationships & interactions
  - ↳ How User, Rider Connected via location
  - ↳ How NotificationService and paymentGateway integrate
- Non-functional Concerns
  - ↳ Data security: protect personal info
  - ↳ Scalability: Architect code to handle millions of users without performance collapse
- Then Apply DSA
  - ↳ Embed Shortest path Algorithm & driver-matching heap inside this object oriented framework

* Core principles of LLD & focus areas

① Scalability.
- ↳ Handle large users volumes easily
- ↳ Code Structure should allow rapid, low-effort expansion (adding server features)

② Maintainability
- ↳ New features shouldn't break existing ones
- ↳ Code should be easy to debug and locate bug.

③ Reusability

↳ Write loosely coupled, "plug & play" modules (eg. generic notification or matching algorithms usable across apps like ~~Zomato~~ Zomato, Swiggy, amazon delivery).

* What LLD is Not (vs HLD)

↳ High level Design (HLD) focuses on System-architecture not code structure

↳ Tech Stack: Choice of languages / frame works (Eg (Javascript, Java. Spring Boot)

↳ Database: SQL vs NoSQL vs hybrid

↳ Server Scaling and deployment: AutoScaling, load balancers, cost optimization on AWS/(GCP).

↳ Cost Considerations: Minimizing Cloud/server expenses per load